Step 6 - Bronze to Silver to Gold

Phase B · The Full Medallion Build · ~25 min

> **Goal:** The portfolio centrepiece - one notebook building all three Medallion layers on your serverless setup: raw Bronze with audit columns, PIPEDA-masked Silver with a quality report, Gold Customer 360 + daily branch summary + FINTRAC LCTR report. Artifact: 05_bronze_silver_gold.ipynb

LEARN

Layer	Rule	MapleBank meaning
Bronze	Store exactly what arrived + audit metadata. Never fix anything here.	Raw CSVs as Delta; SINs still clear text - access-restricted in a real bank
Silver	Cleanse, dedupe, validate, MASK PII. Single source of truth.	SIN last-3, postal to FSA, raw PII columns DROPPED, quality report each run
Gold	Business-ready answers.	Customer 360 (1 row/customer), daily branch summary, FINTRAC LCTR

Masking at the Bronze-Silver boundary means everything downstream is PIPEDA-safe by construction. **The business_date widget** is how ADF parameterizes the notebook in production; standalone it defaults to today.

LAB — notebook 05_bronze_silver_gold (in /MapleBank/, Serverless)

Cell 1 (alone): %run ./utils/masking_functions

Cell 2 — Config + parameter:

```
import pyspark.sql.functions as F
from pyspark.sql.types import (StructType, StructField, StringType,
                               DoubleType, DateType, TimestampType)
from datetime import date

VOL    = "/Volumes/workspace/default/maplebank"
SOURCE = VOL
BRONZE, SILVER, GOLD = f"{VOL}/bronze", f"{VOL}/silver", f"{VOL}/gold"

try:
    dbutils.widgets.text("business_date", "")
    BUSINESS_DATE = dbutils.widgets.get("business_date") or str(date.today())
except:
    BUSINESS_DATE = str(date.today())
print(f"EOD pipeline - business_date = {BUSINESS_DATE}")
```

Cell 3 — the four explicit schemas (same as Step 1).

Cell 4 — BRONZE (raw + audit metadata only):

```
df_raw_customer = spark.read.option("header",True).schema(customer_schema).csv(f"{SOURCE}/dim_customer.csv")
df_raw_account  = spark.read.option("header",True).schema(account_schema).csv(f"{SOURCE}/dim_account.csv")
df_raw_branch   = spark.read.option("header",True).schema(branch_schema).csv(f"{SOURCE}/dim_branch.csv")
df_raw_txn      = spark.read.option("header",True).schema(txn_schema).csv(f"{SOURCE}/fact_transactions.csv")

df_b_customer = add_audit_columns(df_raw_customer, "dim_customer.csv", BUSINESS_DATE)
df_b_account  = add_audit_columns(df_raw_account,  "dim_account.csv",  BUSINESS_DATE)
df_b_branch   = add_audit_columns(df_raw_branch,   "dim_branch.csv",   BUSINESS_DATE)
df_b_txn      = add_audit_columns(df_raw_txn, "fact_transactions.csv", BUSINESS_DATE)

df_b_customer.write.format("delta").mode("overwrite").save(f"{BRONZE}/customers")
df_b_account .write.format("delta").mode("overwrite").save(f"{BRONZE}/accounts")
df_b_branch  .write.format("delta").mode("overwrite").save(f"{BRONZE}/branches")
df_b_txn     .write.format("delta").mode("overwrite") \
             .partitionBy("transaction_date").save(f"{BRONZE}/transactions")
```

Cell 5 — SILVER customers (mask via shared utils, DROP raw PII):

```
df_s_customer = (
    spark.read.format("delta").load(f"{BRONZE}/customers")
    .dropDuplicates(["customer_id"])
    .filter(F.col("customer_id").isNotNull())
    .withColumn("sin_masked",      mask_sin("sin"))
    .withColumn("postal_code_fsa", postal_to_fsa("postal_code"))
    .withColumn("email_masked",    mask_email("email"))
    .withColumn("province",        F.upper(F.trim(F.col("province"))))
    .withColumn("age_years",
        F.floor(F.datediff(F.current_date(), F.col("date_of_birth")) / 365.25))
    .withColumn("age_segment",
        F.when(F.col("age_years") < 25, "YOUNG_ADULT")
         .when(F.col("age_years") < 40, "MILLENNIAL")
         .when(F.col("age_years") < 60, "MID_LIFE")
         .otherwise("SENIOR"))
    .drop("sin","email","phone","street_address","postal_code")  # PII GONE
    .withColumn("_silver_timestamp", F.current_timestamp())
)
df_s_customer.write.format("delta").mode("overwrite").save(f"{SILVER}/customers")
```

Cell 6 — SILVER accounts + branches: dedupe on key, mask_account(), validate_transit(), drop raw account_number; branches pass through with province standardization. Both written to Delta.

Cell 7 — SILVER transactions + quality report:

```
df_b_txn_r = spark.read.format("delta").load(f"{BRONZE}/transactions")
df_s_txn = (
    df_b_txn_r.dropDuplicates(["transaction_id"])
    .filter(F.col("transaction_id").isNotNull()
            & F.col("customer_id").isNotNull()
            & F.col("amount_cad").isNotNull() & (F.col("amount_cad") > 0))
    .withColumn("amount_cad", F.round(F.col("amount_cad"), 2))
    .withColumn("fintrac_flag",
        F.when(F.col("amount_cad") >= 10000, "LCTR_REPORTABLE")
         .when(F.col("amount_cad") >= 9000,  "STRUCTURING_RISK")
         .otherwise("NORMAL"))
    .withColumn("merchant_name", F.coalesce(F.col("merchant_name"), F.lit("UNKNOWN")))
    .withColumn("_silver_timestamp", F.current_timestamp())
)
df_s_txn.write.format("delta").mode("overwrite") \
    .partitionBy("transaction_date").save(f"{SILVER}/transactions")

b, s_ = df_b_txn_r.count(), df_s_txn.count()
print(f"Bronze {b:,}  Silver {s_:,}  Dropped {b-s_:,} ({(b-s_)/b*100:.2f}%)")
df_s_txn.groupBy("fintrac_flag").count().show()
```

Cell 8 — GOLD Customer 360:

```
df_txn_agg = df_s_txn.groupBy("customer_id").agg(
    F.count("*").alias("total_txn_count"),
    F.round(F.sum("amount_cad"),2).alias("total_txn_value_cad"),
    F.max("transaction_timestamp").alias("last_txn_timestamp"),
    F.sum(F.when(F.col("fintrac_flag")=="LCTR_REPORTABLE",1).otherwise(0)).alias("lctr_count"))

df_acct_agg = df_s_account.groupBy("customer_id").agg(
    F.count("*").alias("total_accounts"),
    F.round(F.sum("balance_cad"),2).alias("total_balance_cad"),
    F.collect_set("account_type").alias("product_list"))

df_360 = (df_s_customer
    .join(df_txn_agg, "customer_id", "left")
    .join(df_acct_agg, "customer_id", "left")
    .withColumn("customer_value_segment",
        F.when(F.col("total_txn_value_cad") >= 100000, "PLATINUM")
         .when(F.col("total_txn_value_cad") >= 50000,  "GOLD")
         .when(F.col("total_txn_value_cad") >= 10000,  "SILVER")
         .otherwise("STANDARD")))
df_360.write.format("delta").mode("overwrite").save(f"{GOLD}/customer_360")
```

Cell 9 — GOLD daily branch summary (broadcast the dim) + FINTRAC LCTR report (filter LCTR_REPORTABLE, join MASKED customer fields only, add report_date/report_type literals). Cell 10 — run summary loop over all layers/tables, then:

```
dbutils.notebook.exit(f"SUCCESS: business_date={BUSINESS_DATE}")  # ADF handshake
```

COMMIT + CHECKPOINT

Export to databricks/. Pass criteria: quality report shows small drop pct; customer_360 = 1,000 rows exactly (one per customer); LCTR report ~300 rows; every LCTR column already masked because Silver guarantees it by construction.